IBM Innovation Center for Education

PHEMESOFT Software That Matters

TRUSTED DELIVERY PARTNER for IBM ICE

THE YENEPOYA INSTITUTE OF ARTS,

SCIENCE,COMMERCE & MANAGEMENT (YIASCM)

(a constituent unit of Yenepoya Deemed to be University)

Encrypt and Decrypt data using simple cipher

PROJECT SYNOPSIS

MASTER OF COMPUTER APPLICATION

SUBMITTED BY:

Kapsi Mohammed Aftab 24MCA212

Adithya Ganesh 24MCA201

Geethu Unni B 24MCA207

GUIDED BY:

Mr. Shashank

TITLE PAGE

1. Name of the student: Kapsi Mohammed Aftab
2. Class Roll No. 24MCA212
3. Present official Address: YIASCM Balmatta, Mangalore 575002
4. Branch : Computer Science
5. Batch : 2025-26
6. Proposed Topic : Encrypt and Decrypt data using a simple cipher

TABLE OF CONTENT

INTRODUCTION AND OVERVIEW

1.1 Introduction

This project aims to develop an educational web-based tool that demonstrates the Caesar Cipher, one of the oldest and simplest encryption techniques. Users will be able to interact with the system through a browser, providing input text and a shift value to either encrypt or decrypt messages. The project serves as a gateway to understanding basic cryptographic concepts and the importance of securing information.

1.2 Key Features

- ✓ **Caesar Cipher Only: Simple letter-shift encryption.**
- ✓ **Bidirectional Functionality: Encrypt and decrypt options.**
- ✓ **Web Interface: Built with HTML, CSS, JavaScript.**
- ✓ **Shift Key Support: User-defined numeric key for character shifting.**
- ✓ **Output Display: Shows processed result immediately.**

1.3 Technology Stack

- ✓ **Frontend:** HTML, CSS, JavaScript.
- ✓ **Framework:** Tailwind css.
- ✓ **Backend:** Python (Flask).
- ✓ **Cipher Logic:** Custom Caesar cipher implementation in Python.

1.4 Specialized Field: Cryptography and Cybersecurity

This project introduces students to classical cryptography under the broader domain of cybersecurity, emphasizing how even basic techniques laid the groundwork for modern encryption.

DEVELOPMENT METHODOLOGY

2.1 Methodology

- ✓ Structured in iterative cycles for Caesar cipher implementation.
- ✓ Web interface and backend integration testing.
- ✓ Documentation of Caesar cipher behavior, edge cases, and analysis.

2.2 Requirement Analysis

- ✓ Define cipher logic for Caesar.
- ✓ Identify cryptographic strengths and limitations of Caesar cipher.
- ✓ Ensure accurate input/output handling for diverse data.

2.3 System Design and Architecture

- ✓ Web-based interaction using HTML, CSS, and JavaScript.
- ✓ Caesar cipher logic implemented in Python backend using Flask.
- ✓ Optional module for comparison with AES using PyCryptodome.

2.4 Frontend Development

- ✓ Text areas for plaintext input and encrypted/decrypted output.
- ✓ Input field for shift value (cipher key).
- ✓ Buttons for encryption and decryption actions.
- ✓ Responsive and user-friendly layout using CSS.

2.5 Backend Implementation

- ✓ Python function for Caesar cipher (both encryption and decryption).
- ✓ Handling of edge cases (non-alphabetic characters, case sensitivity).
- ✓ Flask routes for receiving input and returning processed output.

2.6 Final Testing and Documentation

- ✓ Functionality testing for Caesar cipher with various input types.
- ✓ Web usability testing across different browsers and devices.
- ✓ Academic documentation including user manual and technical report.

IMPLEMENTATION AND DEPLOYMENT

3.1 Facilities Required

- ✓ Python 3.12 or later.
- ✓ Code editor or IDE (e.g., VSCode or PyCharm).
- ✓ Internet access for accessing documentation and project deployment.

3.2 Development Environment

- ✓ **IDE**: VSCode / PyCharm.
- ✓ **OS**: Windows/Linux.
- ✓ **Browser:** Chrome, Firefox, or any modern web browser.

3.3 Cryptographic Libraries

- ✓ Standard Python libraries (no external crypto libraries required for Caesar cipher).

3.4 Testing and Deployment

- ✓ Unit testing for Caesar cipher functions.
- ✓ Frontend-backend integration testing using Flask and browser-based testing.
- ✓ Cross-platform testing on both Windows and Linux environments.
- ✓ Local server testing with Flask and optional deployment via tools like Replit or PythonAnywhere.

3.5 Reporting Tools

- ✓ Project documentation using Markdown or Word.
- ✓ PDF/HTML report generation via standard word processing or export tools (e.g., Google Docs, MS Word).